

MonsterCardTradingGame – Protocol Document

1. Introduction

This protocol document summarizes the technical steps, design decisions, encountered failures (and selected solutions), as well as the rationale behind the unit tests for the MonsterCardTradingGame project. It also includes a simple time-tracking overview.

2. Technical Steps & Design Decisions

2.1. Initial Setup & Project Structure

- **Goal:** Create a REST-based game server for a “magical card game” with users, packages, battles, deck management, etc.
- **Technology Stack:**
 - Java (OpenJDK 22)
 - Maven for build management
 - PostgreSQL for persistence

Key Steps:

1. Project Skeleton:

- Created Main.java with a custom HTTP server (Server, Request, Response, Router).
- Established an MVC-like structure with service, controller, repository, and model packages.

2. Database Integration:

- Implemented DatabaseManager, plus UnitOfWork for transaction handling.
- Tables: person, cards, packages, deck, trades, battles, etc.

3. HTTP Endpoints & Handlers:

- Organized around routes: /users, /sessions, /packages, /transactions/packages, /cards, /deck, /battles, /tradings, etc.
- Each endpoint has a dedicated Service class and a corresponding Controller for DB access and logic orchestration.

4. Pure Battle Logic Separation:

- **Reason:** I wanted to keep the “battle rules” (damage calculation, ELO changes, usage-based level-up) separate from DB/HTTP code for easier testing.
- **Solution:** Introduced a BattleLogic class that handles computeDamage, applyElo, incrementUsageAndMaybeLevelUp.

2.2. Failures & Solutions

1. **Issue:** NullPointerException or bound must be positive from Random.nextInt(...)* when decks were empty.
 - **Cause:** One user had zero cards in the deck, causing nextInt(0).
 - **Solution:** Verified deck emptiness right before picking random cards. If empty, the battle ends immediately, awarding a winner.
2. **Issue:** Admin vs. non-admin confusion in /packages creation.
 - **Solution:** Hardcoded admin checks for the user named "admin", or used a boolean isAdmin in DB.
3. **Issue:** Real-time vs. end-of-battle card updates for level-ups.
 - **Solution:** Immediate changes **in memory** (so damage is updated “live” in the fight) but only persist them in the DB after the match ends. This reduces DB traffic.

2.3. Implementation Highlights

- **Unique Feature:** The “level-up” mechanic on cards after every 5 uses, increasing damage by 20 % and level+1, logged in the battle message.
 - **Security:** Token-based checks (e.g., Bearer <token> header) in each route.
-

3. Unit Tests – Rationale & Critical Code

3.1. Why These Unit Tests?

- **Core Combat Rules** are critical because they define how the entire game operates:
 - Goblin vs. Dragon => 0 damage, Wizzard vs. Ork, Knight vs. WaterSpell, etc.
 - Element advantage (fire → normal => x2, water → fire => x2, etc.).
 - These are high-impact rules: if they fail, the entire game’s outcome is incorrect.
- **Usage/Level-up Feature** is unique and can easily break if usage counting is off. Testing it ensures consistency:
 - Level-ups happen exactly after multiples of 5 uses.
 - The +20 % damage logic is easily prone to arithmetic or off-by-one errors.
- **ELO** changes (+3 for winner, -5 for loser) are crucial for the user’s progression and scoreboard.
 - If ELO is miscalculated, the scoreboard becomes meaningless.

3.2. How the Tests Were Structured

- **AAA Pattern** (Arrange, Act, Assert) for each test ensures clarity.
- **No DB Access:** Tests pass plain Card and User objects to BattleLogic, verifying the pure logic.

- **Coverage:** Each special rule is tested (Goblins, Orks, Knights, Kraken, FireElves, etc.), plus the element-based multiplier/halving, usage increments, and ELO updates for multiple consecutive wins/losses.

3.3. Why This Code Is Critical

- The battle, level-up, and ELO logic forms the **core gameplay**. Minor mistakes can lead to major exploits or incorrect results. By thoroughly testing these areas, it is ensured:
 - Players aren't unfairly winning or losing ELO points.
 - The unique level-up feature correctly rewards active usage of cards.
 - Spell- vs. monster-element logic doesn't yield nonsense damage calculations.

4. Time Tracking

Below is a rough estimate of the hours spent, grouped by major tasks:

Task	Time (Hours)
Setting up project skeleton & HTTP server	2.0
Designing DB schema & repositories	13.0
Implementing routes & controllers	21.0
Creating & refining the pure Battle Logic	8.0
Debugging issues (token checks, deck empty)	13.0
Implementing & verifying the unique feature (Level-up)	11.0
Writing & refactoring ~20 Unit Tests	5.0
Final code reviews & integration test checks	2.0
Documentation & protocol writing	1.5
Total	76.5 hours

Conclusion

This protocol demonstrates the main **technical** and **design** steps taken to implement the MonsterCardTradingGame. It shows how **failures**, **solutions**, and **testing** strategy, particularly the separation of the **BattleLogic** to allow for DB-free unit tests was handled, covering **core** combat mechanics and the unique **level-up** feature. The time tracking highlights how the effort was distributed among different project areas.

With this, the project meets the specified requirements:

1. REST-based server with user, package, deck, battle, trade endpoints.
2. A pure BattleLogic for combat & ELO mechanics.
3. A set of 20+ unit tests verifying the critical rules & flows, following the AAA pattern.
4. Documentation capturing the rationale and solutions for encountered challenges.

Note: I have made small changes in terms of HTTP Statuses from those in the specifications, every time when I felt that some other status would be more suitable for a certain situation, e.g. HTTPStatus NO_CONTENT (204) for an empty list, instead of just HTTPStatus OK (200).